

100 Days of Linux - Day 068

Title: Organizing Scripts with Bash Functions

Today's Goal

Learn how to create, call, and reuse functions in Bash scripts.

Why This Matters

Functions help you avoid repeating code. Large automation scripts use functions to keep code organized, readable, and easier to maintain.

Concepts of the Day

Create a Function:

```
greet() {  
    echo "Hello, Linux!"  
}
```

Call the Function:

```
greet
```

Function with Argument:

```
welcome() {  
    echo "Welcome, $1"  
}  
welcome Vishal
```

Beginner Lesson

A function is a reusable block of commands. Instead of writing the same code multiple times, write it once inside a function and call it whenever you need it.

Practice Questions

1. Create a script named `functions.sh` with a function named `greet` that prints 'Welcome to Linux'.
2. Call the `greet` function twice from the same script.
3. Create another function named `show_date` that displays the current date using the `date` command.
4. Create a `welcome` function that accepts a name as an argument and prints 'Hello, NAME!'. Test it with your own name.
5. Run the script and display your current working directory.

Hints

Define functions before calling them. Pass arguments just like you pass arguments to a script.

Mini Challenge

Create `system_report.sh` with three functions: `show_user`, `show_directory`, and `show_date`. Call all three to generate a simple system report.

Common Mistakes

Calling a function before it is defined, or adding spaces between the function name and parentheses.

Did You Know?

Many production Bash scripts contain dozens of functions to separate configuration, validation, logging, and deployment tasks.

Pro Tip

Keep each function focused on a single task. Small, reusable functions make debugging much easier.

Answer Key (Instructor Only)

Q1: `greet(){ echo 'Welcome to Linux'; }`

Q2: `greet && greet`

Q3: `show_date(){ date; }`

Q4: `welcome(){ echo "Hello, $1!"; } && welcome Vishal`

Q5: `bash functions.sh && pwd`

Homework

Create a script with at least four functions: `welcome`, `show_user`, `show_date`, and `show_directory`. Call each function in order.

Next Day Preview

Tomorrow you'll combine variables, user input, conditions, loops, and functions to build a complete interactive Bash application.